

# **Huffman & AEP Recap + something new!**

**EE274**

**Kedar Tatwawadi**

# Huffman coding - Recap

## Optimal prefix-free code

The problem of finding *optimal prefix-free code* given a probability distribution  $p_1, p_2, \dots, p_k$  can be formulated as:

Given probabilities  $p_1, p_2, \dots, p_k$ , solve for code-lengths  $l_i \in \mathbb{N}$ , such that:

$$\text{minimize } L(p) = \sum_{i=1}^k p_i l_i$$

$$\text{under the constraint } \sum_{i=1}^k 2^{-l_i} \leq 1$$

# Huffman coding - Recap

- Sort the probabilities, and build singleton nodes

```
probs_dict = {"A": 0.35, "B": 0.25, "C": 0.2, "D": 0.12, "E": 0.08}
```



**STEP 0:** The initial step is to first build singleton nodes from these probabilities.

A  
(p=0.35)

B  
(p=0.25)

C  
(p=0.2)

D  
(p=0.12)

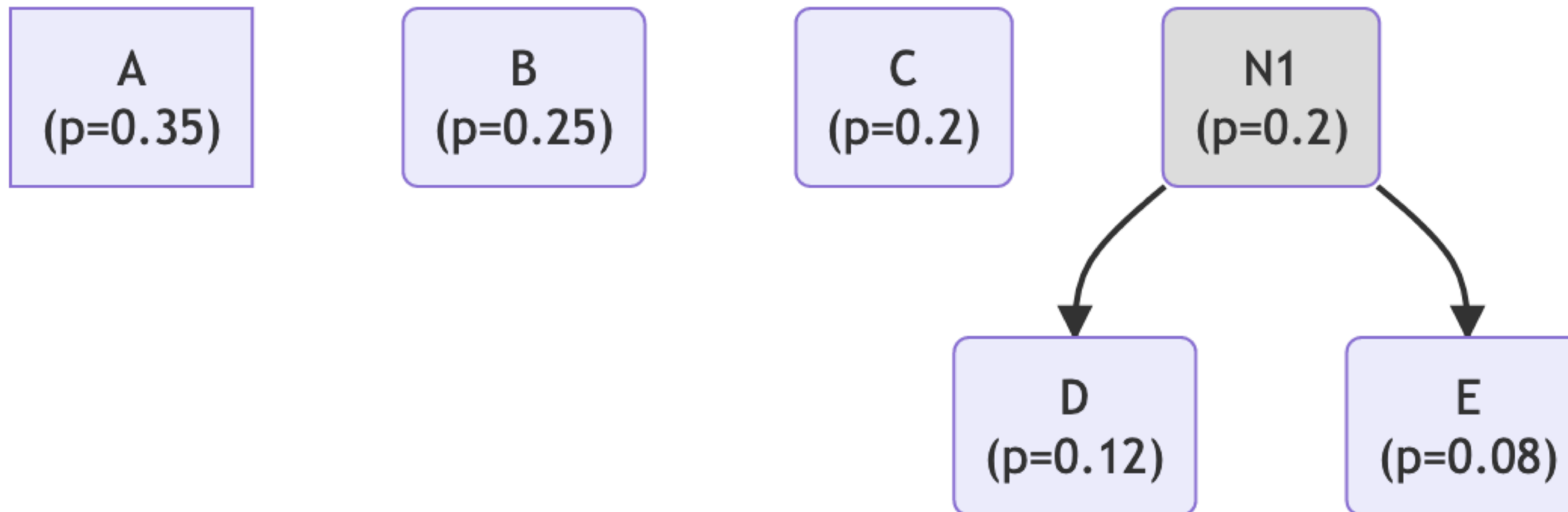
E  
(p=0.08)



# Huffman coding - Recap

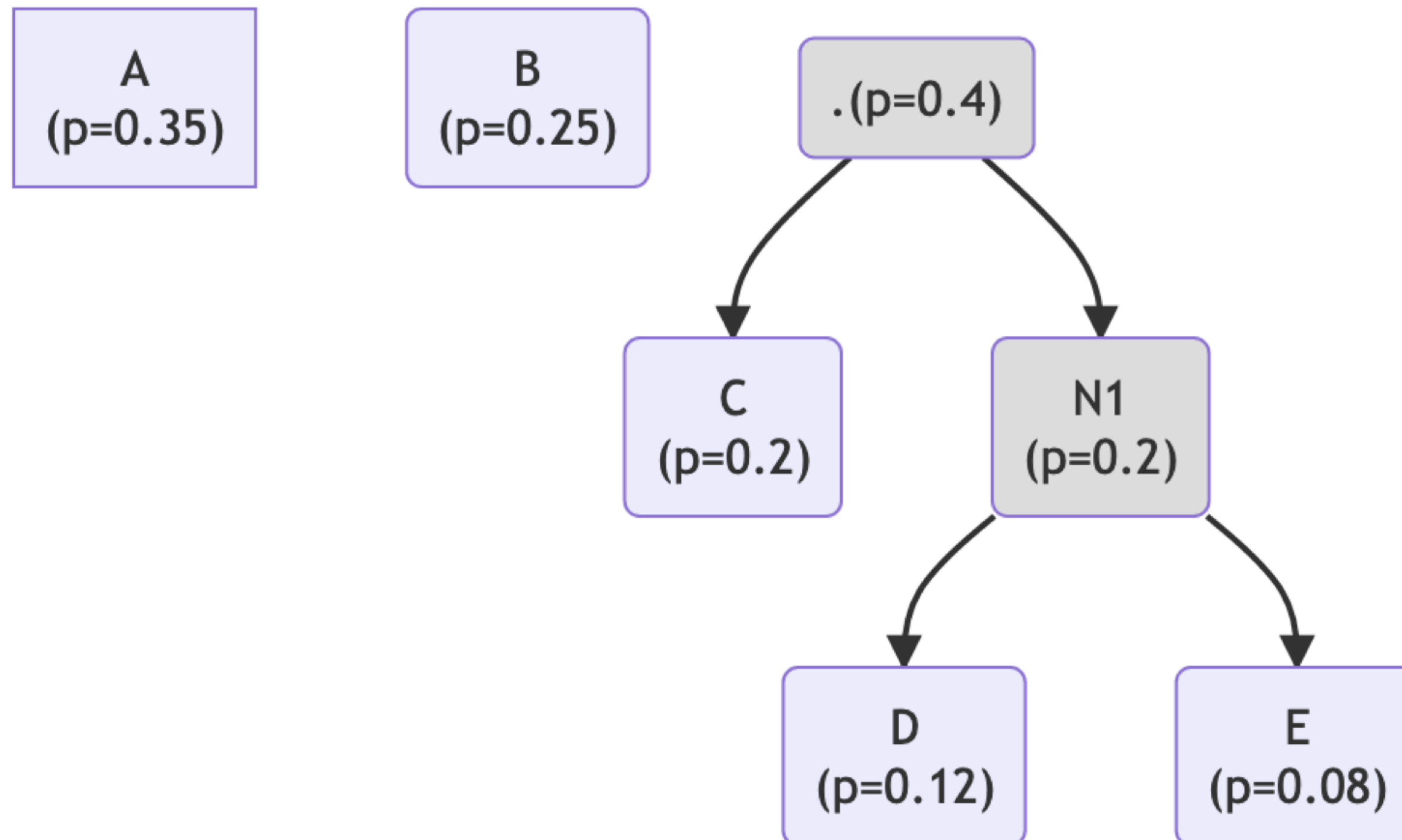
```
## Iter 1
```

```
node_list = [Node(A, 0.35), Node(B, 0.25), Node(C, 0.2), Node(N1, 0.2)]
```



# Huffman coding - Recap

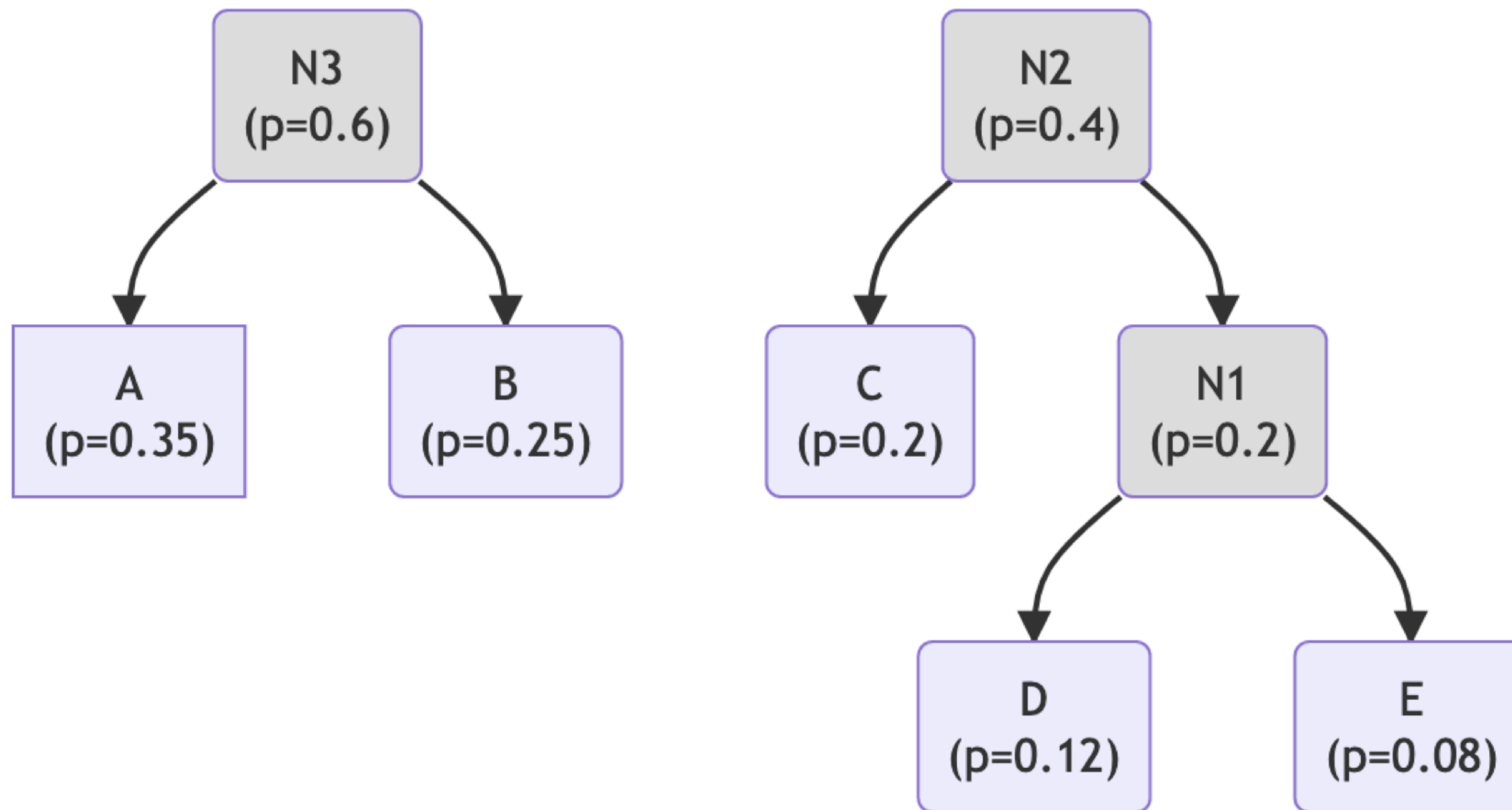
```
## Iter 2  
node_list = [Node(A, 0.35), Node(B, 0.25), Node(N2, 0.4)]
```



# Huffman coding - Recap

## Iter 3

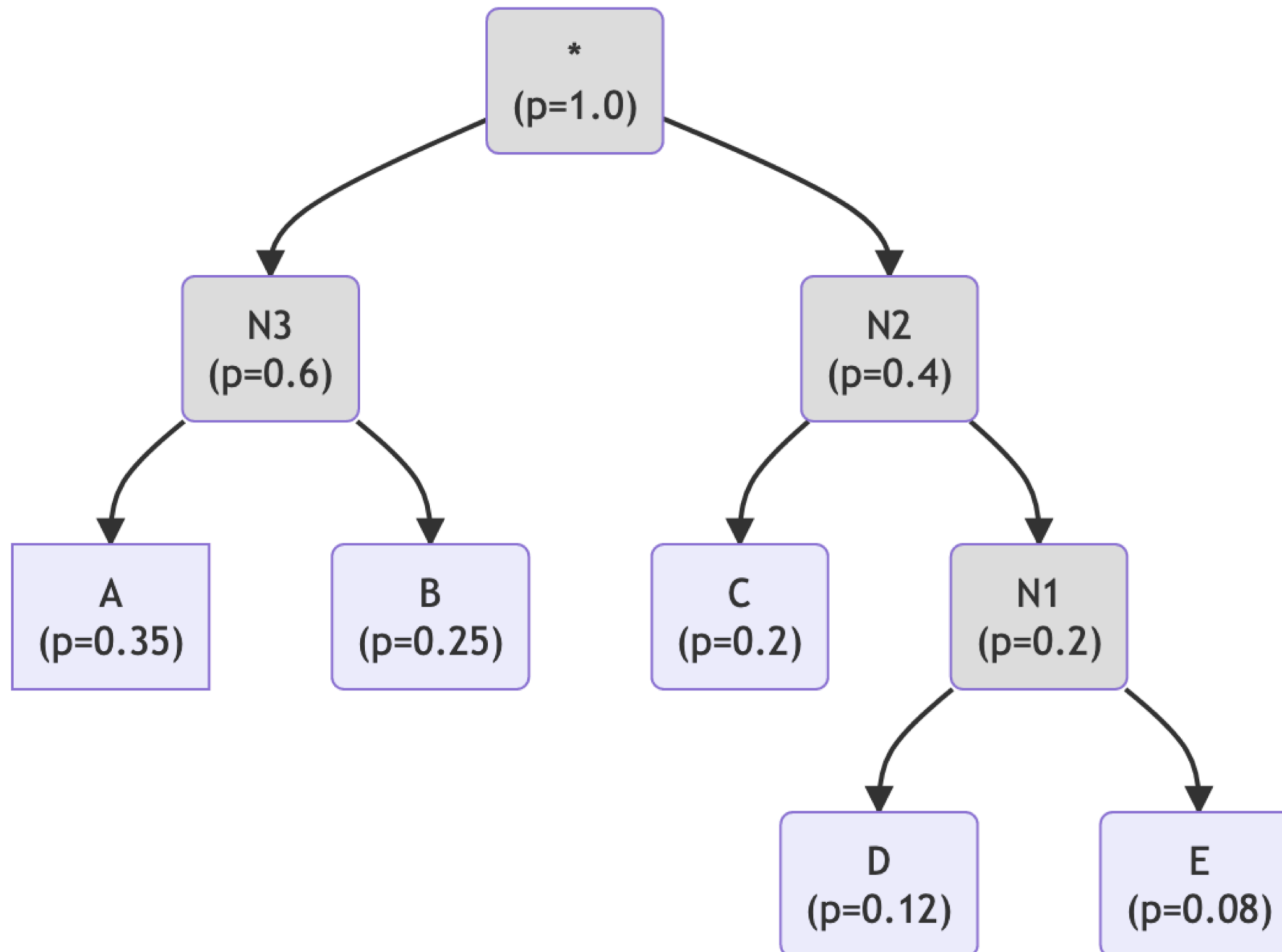
```
node_list = [Node(N2, 0.4), Node(N3, 0.6)]
```





# Huffman coding - Recap

```
## Iter 4  
node_list = [Node("*", 1.0)]
```



# Huffman coding - Recap

## Theorem-8: Necessary conditions for prefix-free code optimality

Any optimal prefix free code for a distribution  $P = \{p_1, p_2, \dots, p_k\}$  must satisfy the following conditions:

**Condition 1** The code-lengths are ordered in inverse order of the probabilities, i.e.

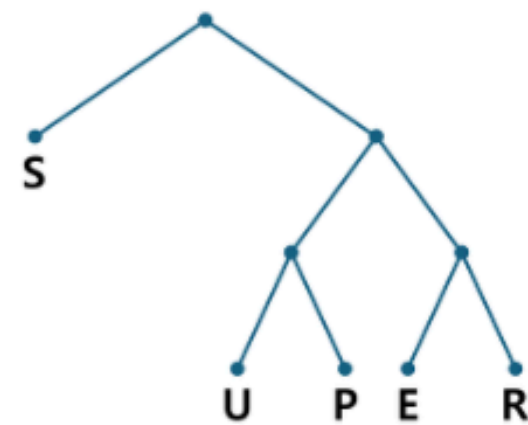
$$p_i > p_j \Rightarrow l_i \leq l_j$$

**Condition 2** The two longest codewords have the same code-lengths

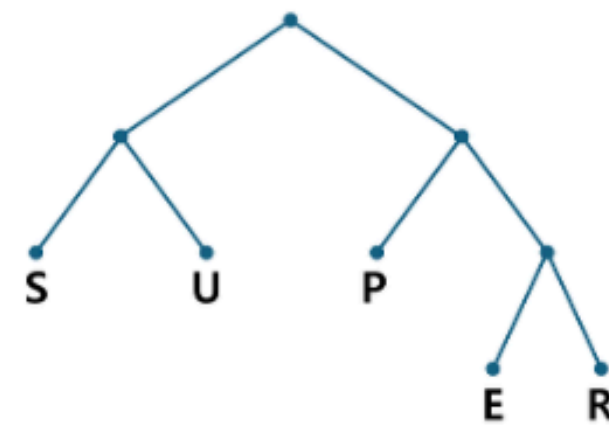


# Huffman coding - Quiz discussion

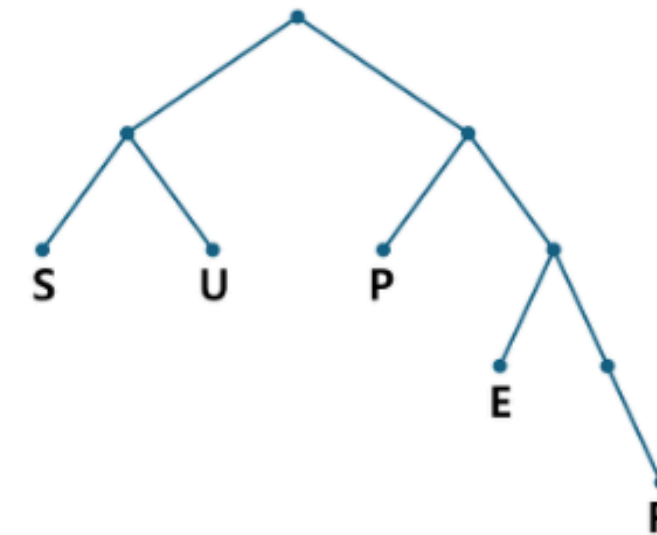
For each of the provided prefix-free codes for the alphabet  $\{S, U, P, E, R\}$  suggest a probability distribution for which the provided code is the Huffman code. If you believe the code cannot be the Huffman code for any distribution, explain why.



Code 1



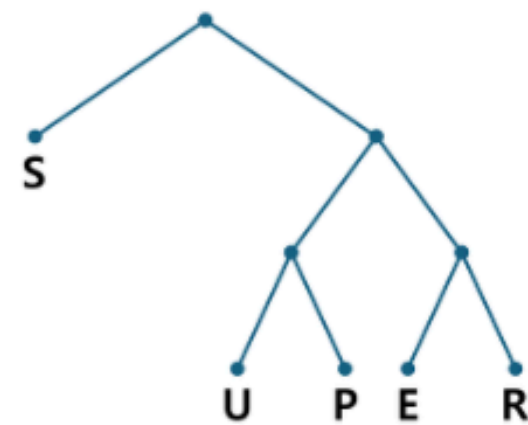
Code 2



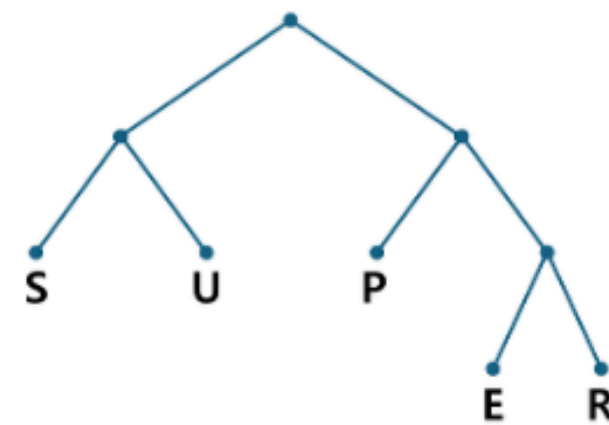
Code 3

# Huffman coding - Quiz discussion

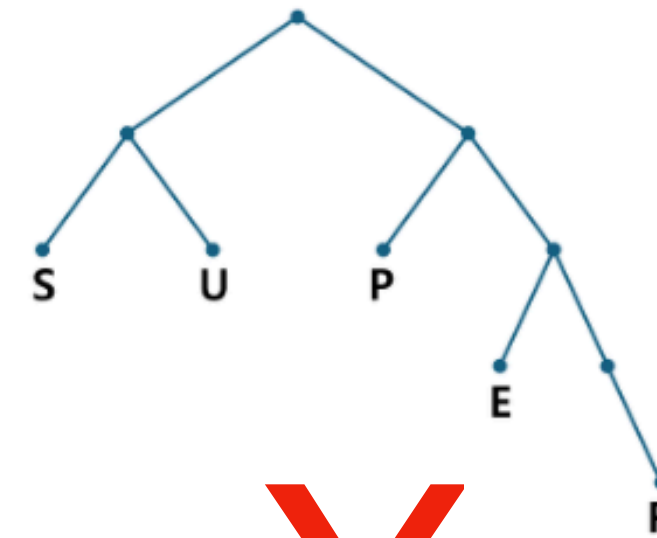
For each of the provided prefix-free codes for the alphabet  $\{S, U, P, E, R\}$  suggest a probability distribution for which the provided code is the Huffman code. If you believe the code cannot be the Huffman code for any distribution, explain why.



Code 1



Code 2



Code 3



# Huffman coding - Quiz discussion

For a given sample  $(X_1, X_2, \dots, X_m) = (x_1, x_2, \dots, x_m)$  from the iid source, will the encoded length for the Huffman code discussed above always exceed  $mH(X)$ ?

Note that we're considering the codelength of a **specific sample**, rather than the expected codelength.



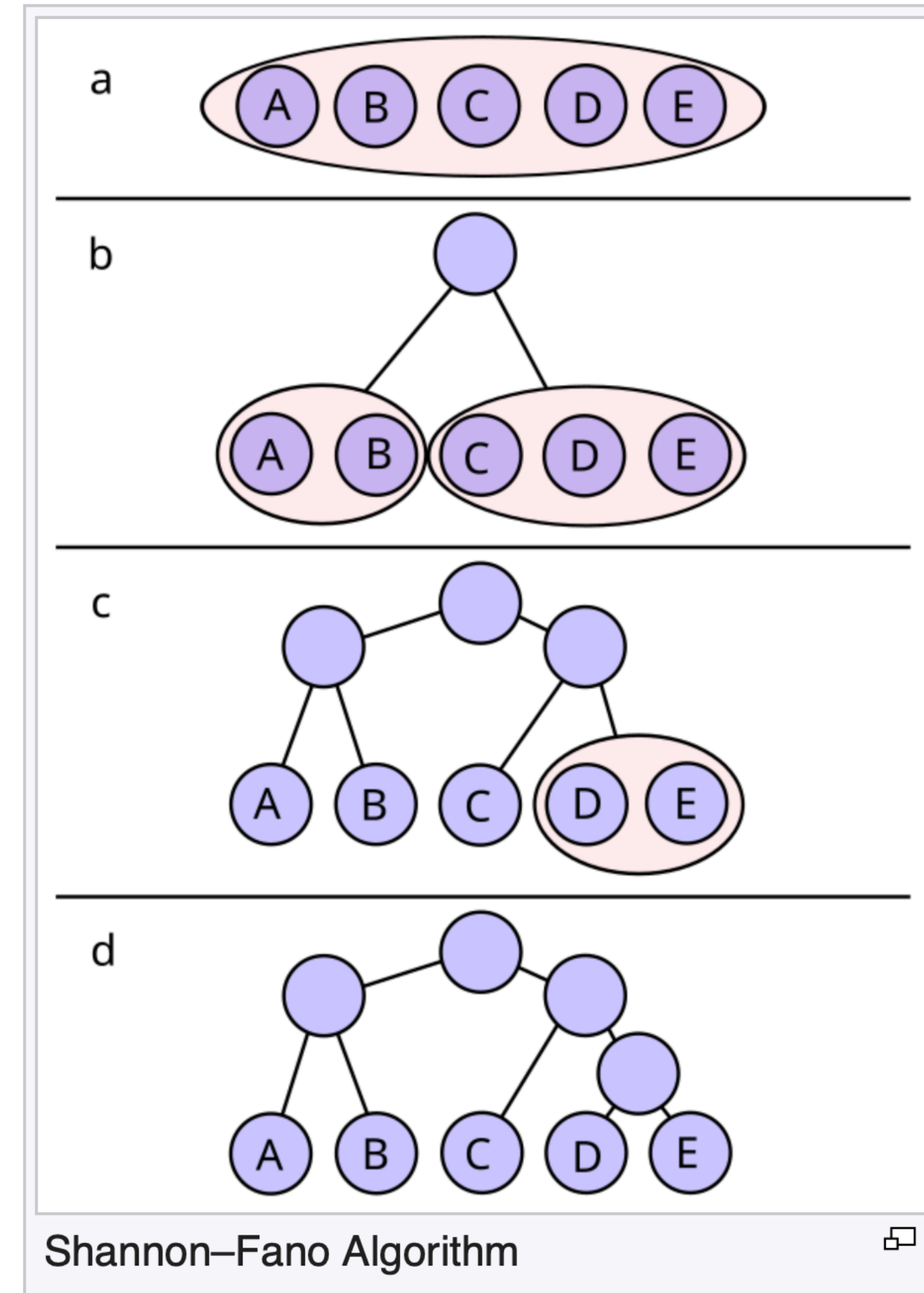
# Being *greedy* can sometimes be good!

- Not all *greedy algorithms* are optimal
- Multiple “greedy” ways to construct prefix-free codes.
  - Fano codes
  - Huffman codes
- Huffman codes can be *proved* to be optimal (not trivial!)

# Fano codes

| Symbol        | A     | B     | C     | D     | E     |
|---------------|-------|-------|-------|-------|-------|
| Count         | 15    | 7     | 6     | 6     | 5     |
| Probabilities | 0.385 | 0.179 | 0.154 | 0.154 | 0.128 |

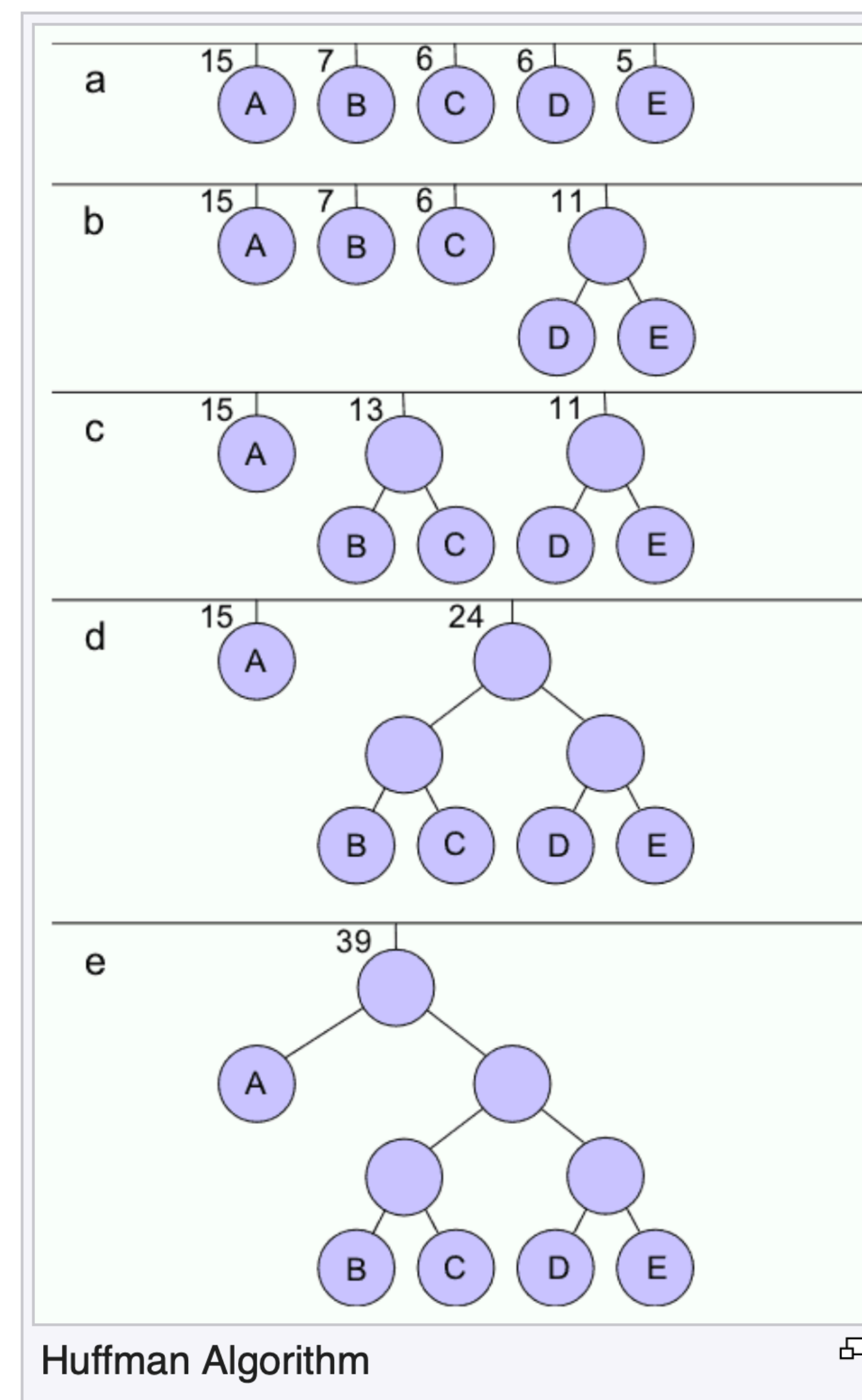
Avg-codelen = 2.28



# Huffman codes

| Symbol        | A     | B     | C     | D     | E     |
|---------------|-------|-------|-------|-------|-------|
| Count         | 15    | 7     | 6     | 6     | 5     |
| Probabilities | 0.385 | 0.179 | 0.154 | 0.154 | 0.128 |

Avg-codelen = 2.23





# Huffman coding - Proof Recap

## Theorem-8: Necessary conditions for prefix-free code optimality

Any optimal prefix free code for a distribution  $P = \{p_1, p_2, \dots, p_k\}$  must satisfy the following conditions:

**Condition 1** The code-lengths are ordered in inverse order of the probabilities, i.e.

$$p_i > p_j \Rightarrow l_i \leq l_j$$

**Condition 2** The two longest codewords have the same code-lengths

The main property is that after the *merge* step of merging two smallest probability nodes of probability distribution  $(p_1, p_2, \dots, p_k)$ , the remaining tree is still optimal for the distribution  $(p_1, p_2, \dots, p_{k-1} + p_k)$ .

See (Cover & Thomas, sec 5.8 for detailed proof)



# Huffman coding in the wild

| Type                | Compressor / Standard      | Use of Huffman Coding                                                   | Reference / Notes              |  |
|---------------------|----------------------------|-------------------------------------------------------------------------|--------------------------------|-------------------------------------------------------------------------------------|
| General-purpose     | DEFLATE (gzip, zlib)       | Dynamic or fixed Huffman codes for literals, lengths, and distances     | RFC 1951                       |                                                                                     |
|                     | ZIP files                  | DEFLATE compression → Huffman codes for literals/distances              | PKWARE Spec                    |                                                                                     |
|                     | Zstandard (Zstd)           | Huffman coding for literal symbols; combined with FSE for other symbols | RFC 8878                       |                                                                                     |
|                     | Brotli                     | Huffman coding applied to literals and lengths in each meta-block       | RFC 7932                       |                                                                                     |
| Images              | PNG                        | Uses DEFLATE internally → Huffman coding for pixel data                 | PNG Spec (ISO/IEC 15948)       |                                                                                     |
|                     | JPEG Baseline              | Huffman coding for quantized DCT coefficients                           | ITU-T T.81 (ISO/IEC 10918-1)   |                                                                                     |
| Video               | H.261                      | Huffman coding for macroblock data                                      | RFC 2032                       |                                                                                     |
|                     | H.263                      | Huffman coding for residuals and motion vectors                         | ITU-T H.263                    |                                                                                     |
|                     | H.264/AVC (CAVLC mode)     | Huffman-like VLCs for residual coefficients in baseline/main profiles   | ITU-T H.264 / ISO/IEC 14496-10 |                                                                                     |
| Documents / Streams | PDF (Flate / Zlib streams) | Flate uses DEFLATE compression → Huffman coding for streams             | PDF 32000-1:2008               |                                                                                     |



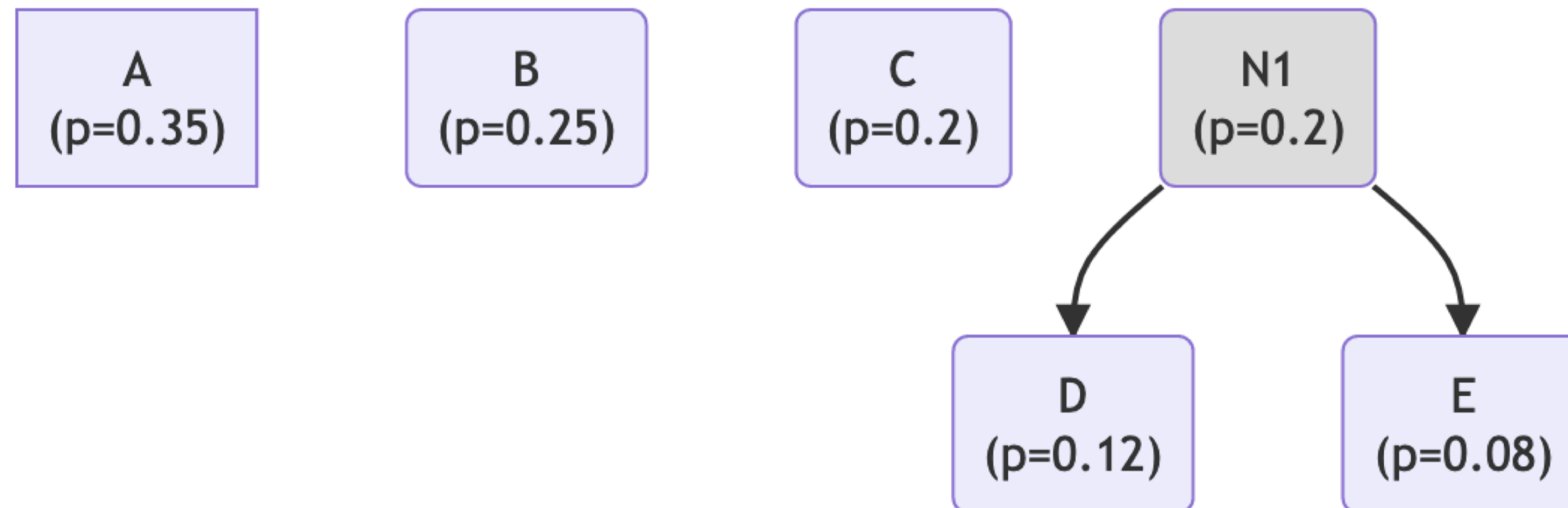
# Huffman coding - practical issues:

- **Issue-1:** Ambiguity in the tree construction
- **Issue-2:** Decoding speed can be slow

# Issue-1: Ambiguity in Huffman code construction

- Which symbols goes to the left/right subtree?
- Which symbol is chosen when both have the same probability?

```
## Iter 1  
node_list = [Node(A, 0.35), Node(B, 0.25), Node(C, 0.2), Node(N1, 0.2)]
```



# Solution: Canonical Huffman codes

- **Step-1:** Huffman tree construction is used to get optimal lengths for each symbol
- **Step-2:** Construct a “canonical” prefix-free tree from those lengths. (i.e. use some simple conventions)

## Appendix B. Huffman Code

The following Huffman code is used when encoding string literals with a Huffman coding (see [Section 5.2](#)).

This Huffman code was generated from statistics obtained on a large sample of HTTP headers. It is a canonical Huffman code (see [\[CANONICAL\]](#)) with some tweaking to ensure that no symbol has a unique code length.



# Aside: Huffman tree inception!

Each block consists of two parts: a pair of Huffman code trees that describe the representation of the compressed data part, and a compressed data part. (The Huffman trees themselves are compressed using Huffman encoding.) The compressed data consists of a series of elements of two types: literal bytes (of strings that have not been detected as duplicated within the previous 32K input bytes), and pointers to duplicated strings, where a pointer is represented as a pair <length, backward distance>. The representation used in the "deflate" format limits distances to 32K bytes and lengths to 258 bytes, but does not limit the size of a block, except for uncompressible blocks, which are limited as noted above.

# Issue-2: Huffman codes decoding speed

- **Key-Idea:** Tree-based decoding creates too many branches -> “precompute the decodings”

```
## faster decoding
state_size = 3
decode_state = {
    000: "A",
    001: "A",
    010: "A",
    011: "A",
    100: "B",
    101: "B",
    110: "C",
    111: "D",
}
codelen_table = {
    "A": 1,
    "B": 2,
    "C": 3,
    "D": 3,
}
def decode_symbol_tree(encoded_bitarray):
    state = encoded_bitarray[:state_size]
    decoded_symbol = decode_state[state]
    num_bits_consumed = codelen_table[decoded_symbol]
    return decoded_symbol, num_bits_consumed
```

# Issue-2: Huffman codes decoding speed

- **Key-Idea:** “precompute the decodings” -> trade-off memory for compute. Memory is exponential in the **max-codelength**.
- **Fix-1:** Limit the maximum depth of the tree (max-codelength)
- **Fix-2:** Use hierarchical trees -> a dummy symbol is used to represent less-frequent symbols. This then leads to another table.

```
## faster decoding
state_size = 3
decode_state = {
    000: "A",
    001: "A",
    010: "A",
    011: "A",
    100: "B",
    101: "B",
    110: "C",
    111: "D",
}
```



# Huffman Codes on blocks - Recap

$P = \{A: 0.1, B: 0.9\}$   
 $H(P) = 0.47$

It is clear that as there are just two symbols, the only reasonable prefix-free code design here is:

$A \rightarrow 0, B \rightarrow 1$   
Huffman code (or any other prefix-free code) = 1 bit



# Huffman Codes on blocks - Recap

$P = \{A: 0.1, B: 0.9\}$   
 $H(P) = 0.47$

block\_size: 1, entropy: 0.47, avg\_codelen: 1.00 bits/symbol

```
  |--A
.-|
  |--B
```

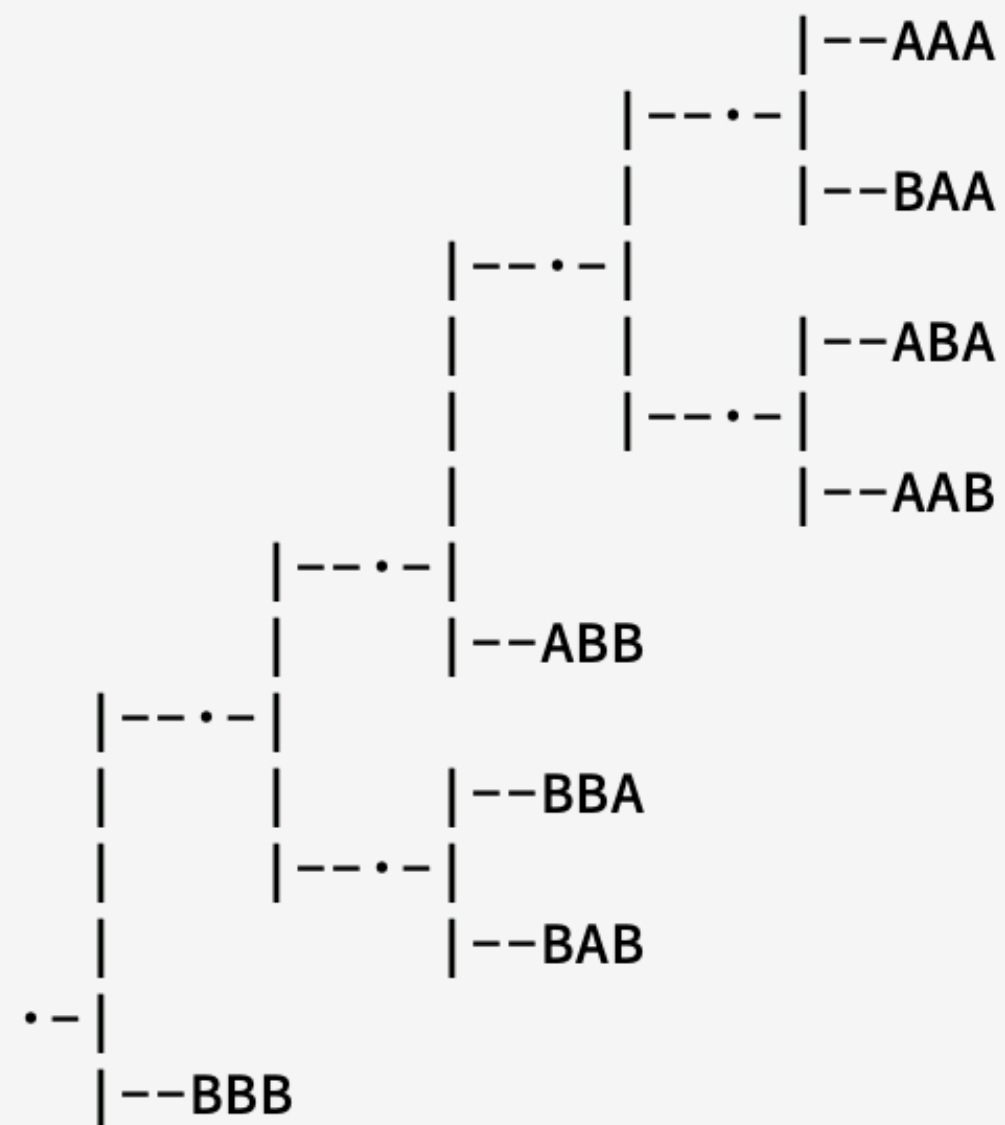
block\_size: 2, entropy: 0.47, avg\_codelen: 0.65 bits/symbol

```
      |--BA
    |--.-|
    |    |--AA
    |    |--.-|
    |    |--AB
.-|
|--BB
```



# Huffman Codes on blocks - Recap

block\_size: 3, entropy: 0.47, avg\_codelen: 0.53 bits/symbol



## Huffman code for blocks

block\_size: 1, entropy: 0.47, avg\_codelen: 1.00 bits/symbol  
block\_size: 2, entropy: 0.47, avg\_codelen: 0.65 bits/symbol  
block\_size: 3, entropy: 0.47, avg\_codelen: 0.53 bits/symbol  
block\_size: 4, entropy: 0.47, avg\_codelen: 0.49 bits/symbol  
block\_size: 5, entropy: 0.47, avg\_codelen: 0.48 bits/symbol

# Huffman Codes on blocks - Recap

1. Huffman codes

$$H(X) \leq \mathbb{E}[l(X)] \leq H(X) + 1$$

2. Huffman codes on blocks of size B

$$H(X) \leq \frac{\mathbb{E}[l(X_1^B)]}{B} \leq H(X) + \frac{1}{B}$$

# AEP - Asymptotic Equipartition Property

## RECAP

### Definition-1

For some  $\epsilon > 0$ , the source sequence  $U^n$  is  **$\epsilon$ -typical** if

$$\left| -\frac{1}{n} \log P(U^n) - H(U) \right| \leq \epsilon$$



# AEP- Recap

## Definition-1

For some  $\epsilon > 0$ , the source sequence  $U^n$  is  **$\epsilon$ -typical** if

$$\left| -\frac{1}{n} \log P(U^n) - H(U) \right| \leq \epsilon$$

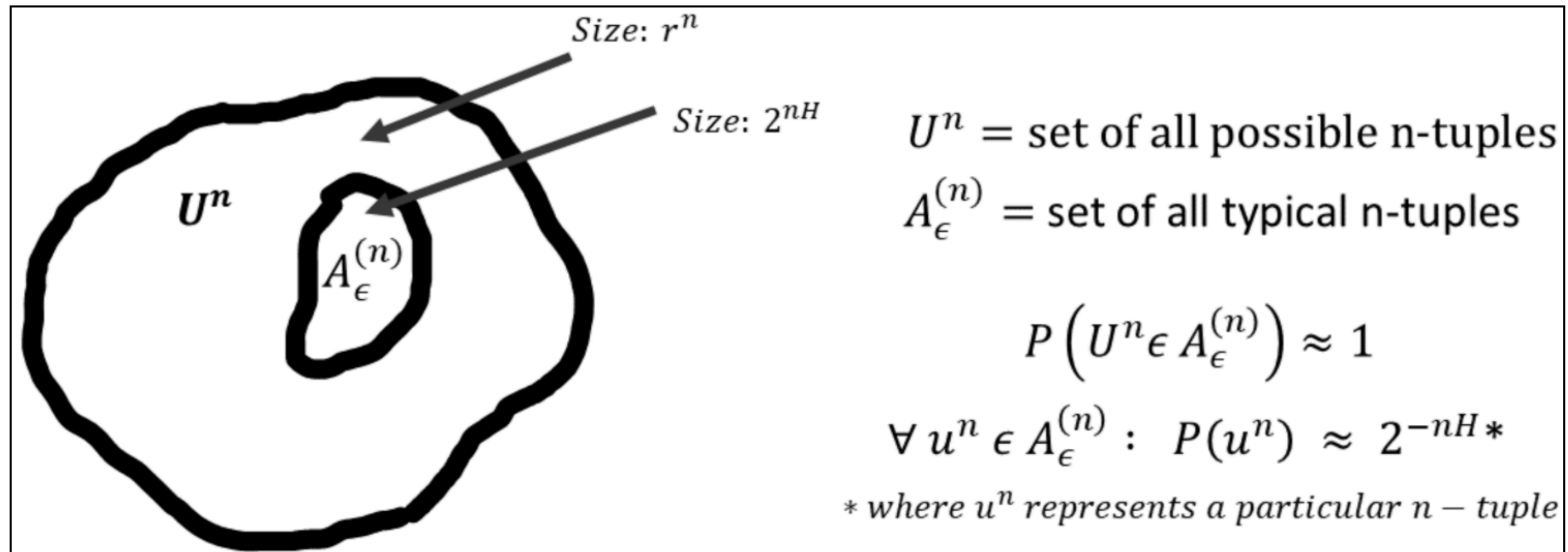
## Theorem-2: Properties of typical sets

For any  $\epsilon > 0$ ,

1.  $\lim_{n \rightarrow \infty} P(U^n \in A_\epsilon^{(n)}) = 1.$
2.  $|A_\epsilon^{(n)}| \leq 2^{n(H(U)+\epsilon)}$
3. For large enough  $n$ ,  $|A_\epsilon^{(n)}| \geq (1 - \epsilon)2^{n(H(U)-\epsilon)}$

Simply put, this is saying that for large  $n$ , the typical set has probability close to 1 and size roughly  $2^{nH(U)}$

# AEP- Recap



# AEP-typical block coder

- **Key-idea-1:** As the block size gets larger, with a very high probability the sequence lies in the typical set.
- For all practical purposes, we can just focus our attention on compressing typical sequences (and do something naive for a-typical sequences)



# AEP- Recap

## EE274 – Asymptotic Equipartition Property (AEP) Visualization

p (Bernoulli parameter): 0.16

$\epsilon$ : 0.05

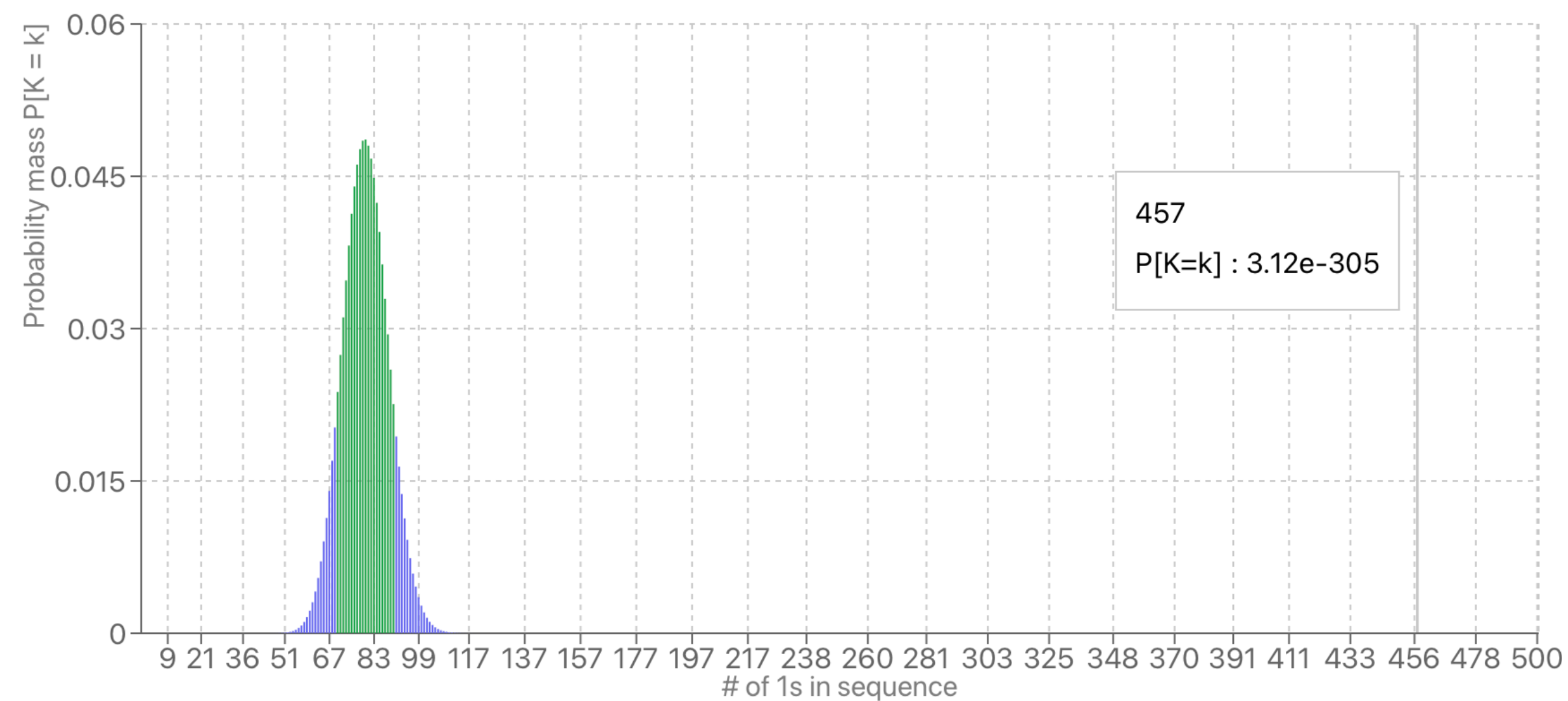
n: 500

Chart mode:

Probability mass

Count fraction

Probability: how much probability the typical set covers. Count: what fraction of all sequences are typical.



**Entropy H:** 0.634 bits. Bars in green denote counts whose per-sequence probability falls within the typical band  $(H \pm \epsilon)$ .

**Total probability of typical sequences:** 80.01%

Tip: For skewed  $p$  and small  $\epsilon$ , the probability mass approaches 1 quickly, but the fraction of sequences in the typical set remains exponentially small.

<https://chatgpt.com/canvas/shared/68dabb29e9b881918131da8137f85696>

# AEP- Quiz

Now switch to the “**Count fraction**” view. Fix  $\epsilon = 0.1$ ,  $n = 500$ . Vary  $p$  from 0.1 to 0.5. What happens to the fraction of typical sequences as you vary  $p$ ? Briefly explain the trend in terms of the AEP.

<https://chatgpt.com/canvas/shared/68dabb29e9b881918131da8137f85696>



# AEP-typical block coder

- **Take-away-1:** As the block size gets larger, with a very high probability the sequence lies in the typical set.
- **Take-away-2:** The entropy limit ( $H(U)$ ) in compression comes from the fact that the size of the typical set is  $2^{\{n \cdot H(U)\}}$ , and that the sequences are almost uniformly distributed.

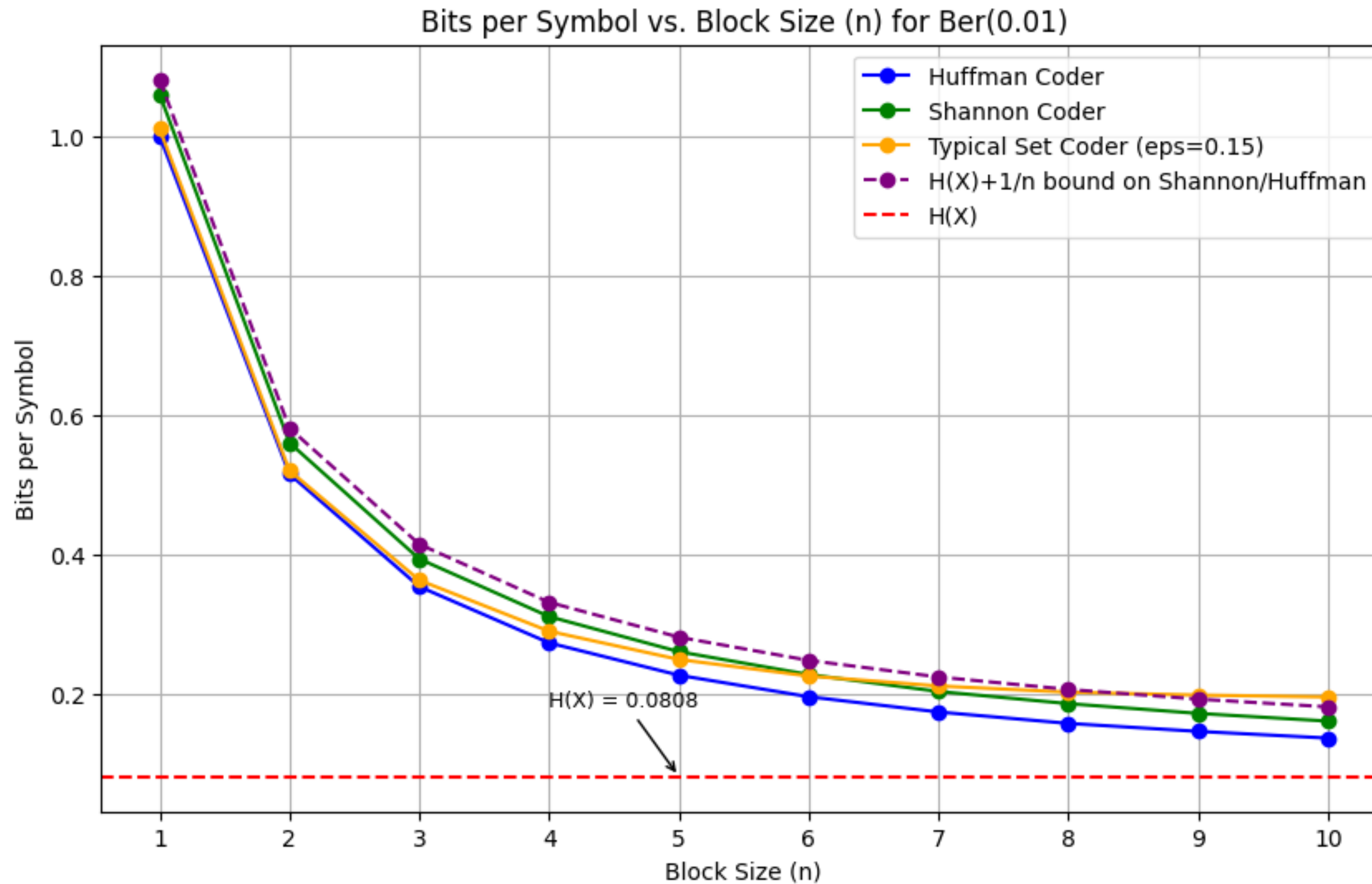
# AEP-typical block coder

## Lossless compression using typical sets

Fix  $\epsilon > 0$ , and assign index  $idx$  ranging from  $1, \dots, |A_\epsilon^{(n)}|$  to the elements in  $A_\epsilon^{(n)}$  (the order doesn't matter). In addition, define a fixed length code  $fixed$  for  $\mathcal{U}^n$  that uses  $n \log_2 |\mathcal{U}|$  bits to encode any input sequence. Now the encoding of  $u^n$  is simply:

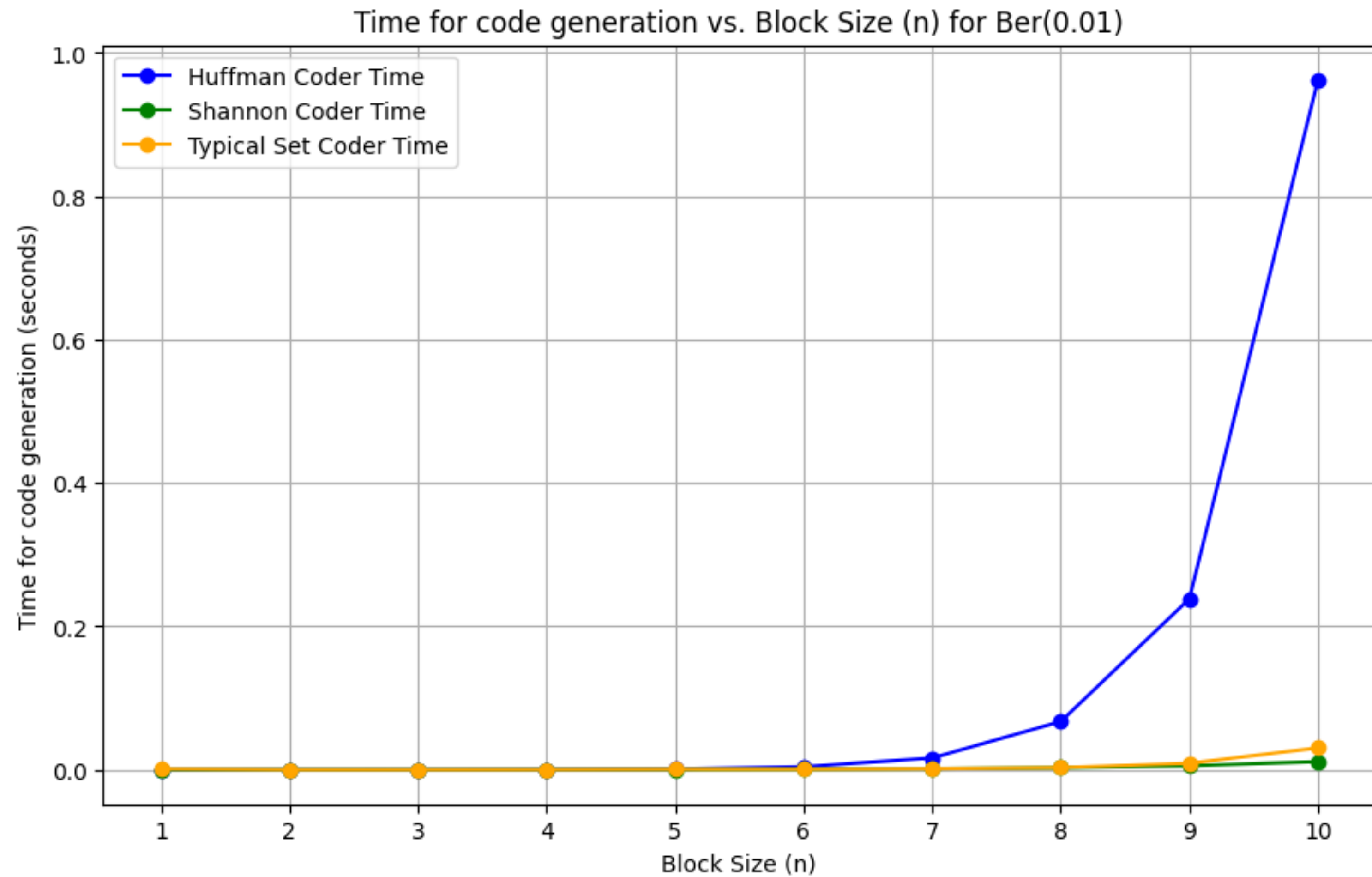
- if  $u^n \in A_\epsilon^{(n)}$ , encode as 0 followed by  $idx(u^n)$
- else, encode as 1 followed by  $fixed(u^n)$

# AEP- Recap





# AEP- Recap



# Universal block coder puzzle!

**Question:** Lets assume binary iid source with  $P(X=1) = p$ .

The typical coder above needs to know  $p$ , to create the codebooks.

Can we create a “universal block coder” which does not need to know  $p$ ?

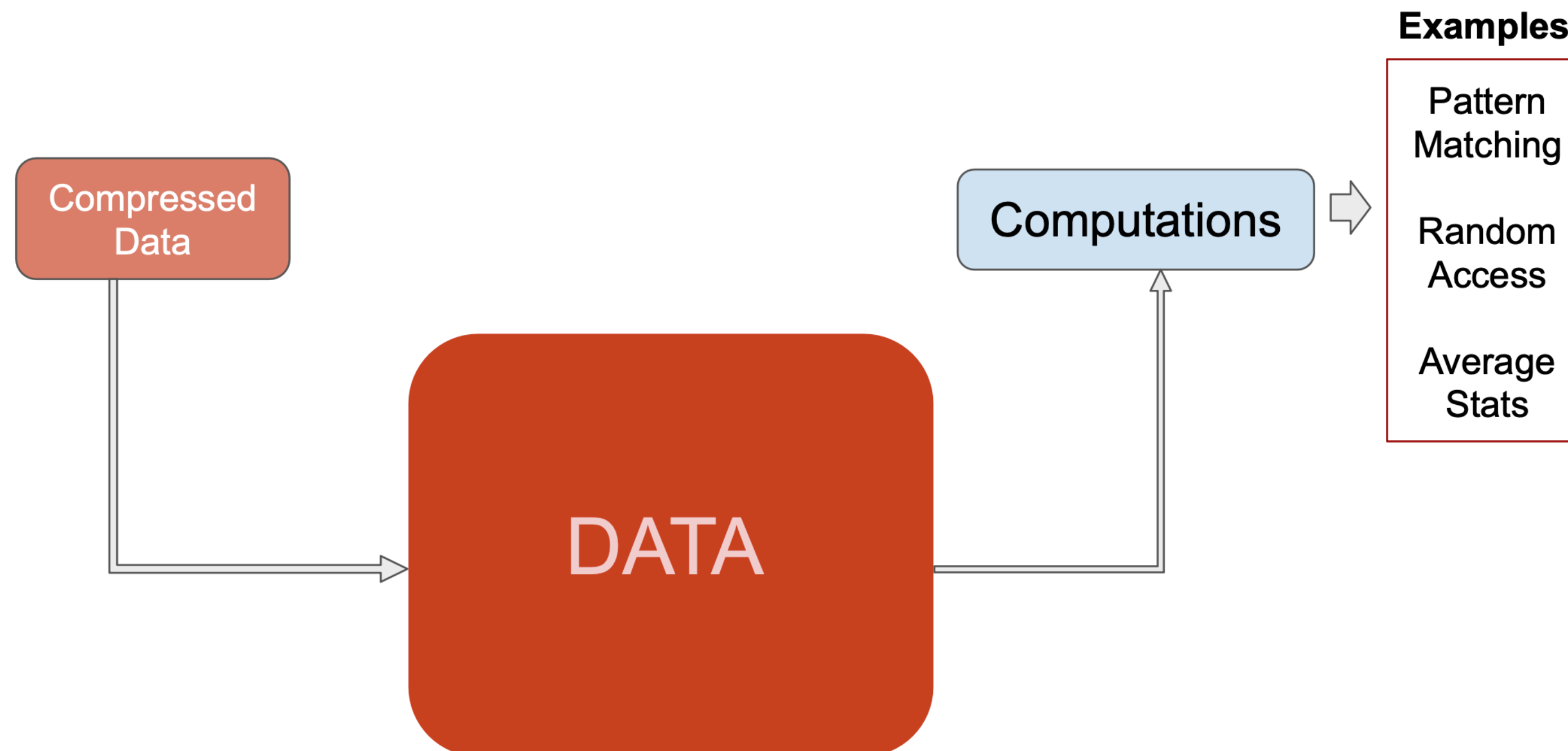
# Going beyond Compression

- So far we have focused on designing compressors which will give us good compression. But, in practice there are other important goals:
  - Random Access /Local decodability  
(decoding a symbol without fully decompressing the data)
  - computation in the compressed domain  
(i.e. without fully decompressing the data)



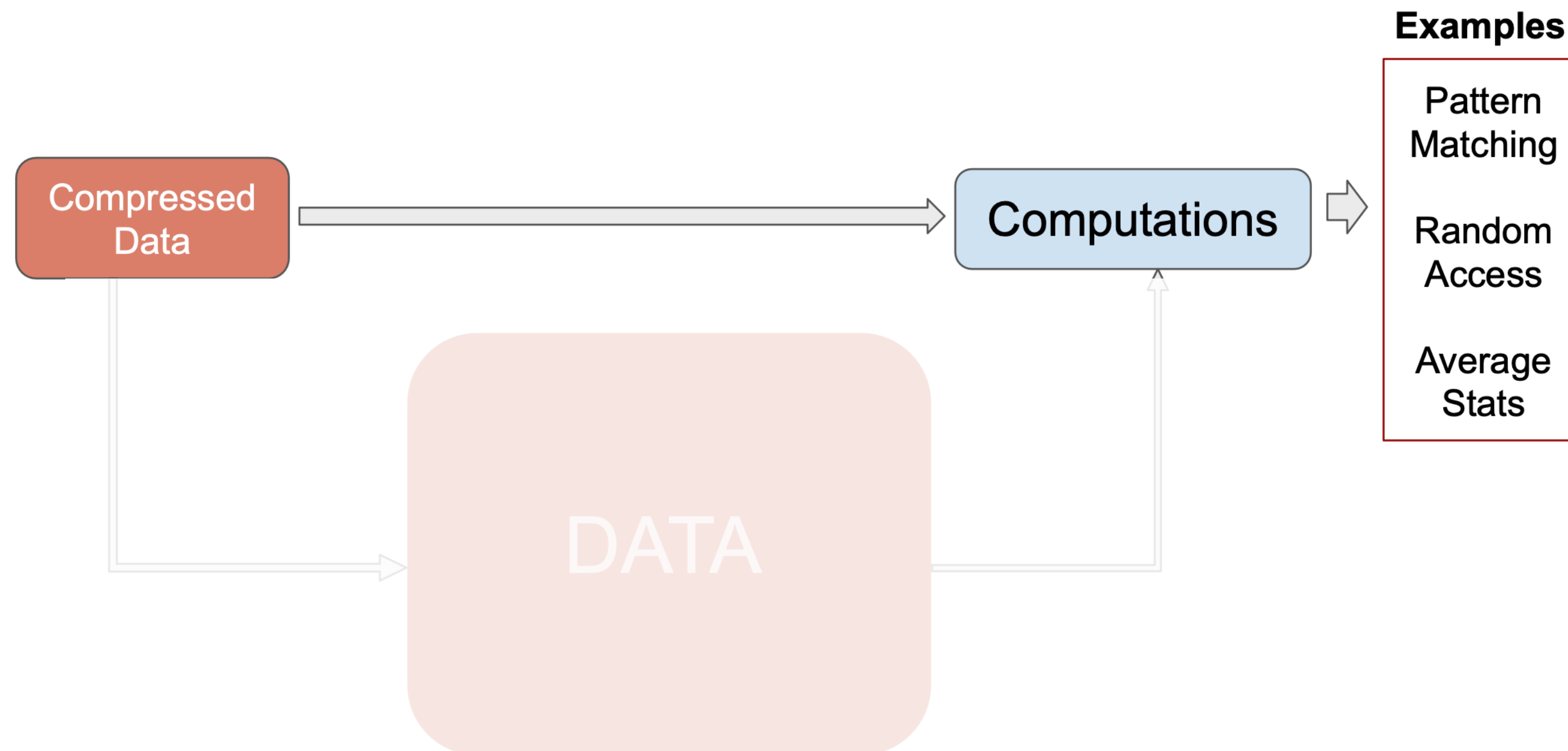
# Succinct data structures:

- Class of algorithms with following properties:
  - compress the data sequence close to optimality (Entropy)
  - support an important functionality efficiently.



# Succinct data structures:

- Class of algorithms with following properties:
  - compress the data sequence close to optimality (Entropy)
  - support an important functionality efficiently.





# Random Access/Local decidability

- **Goal:** to be able to decode the  $k^{\text{th}}$  symbol from the compressed string, without decompressing the full sequence.
- **Question:** Does Huffman coded sequence support random access?

# Random Access/Local decodability

**Question:** Does Huffman coded sequence support random access?

Sample sequence: **A B A A C B A ...**

Code: **0100011100...**

Q1: Can you decode the 5th symbol without decoding the entire sequence?

| Symbol | Code |
|--------|------|
| A      | 0    |
| B      | 10   |
| C      | 11   |

# Random Access/Local decodability

**Question:** How can we modify Huffman codes to support random access?

- **Idea-1:** Cut the sequence into blocks and store a pointer to each block in the encoded
  - *pro*: need to only decode a single block
  - *con*: larger block size -> slower random access  
smaller block size -> worse compression

| Symbol | Code |
|--------|------|
| A      | 0    |
| B      | 10   |
| C      | 11   |

Sample sequence: **A B A A C B A ...**  
Code: **0100011100...**



# Random Access/Local decodability

**Question:** How can we modify Huffman codes to support random access?

- **Idea-2:** What if we *magically know* the number of A,B,C's until a given position?

$\text{rank}(\text{bitstream}, \text{pos}=5, \text{sym}=\text{B}) = 1$

- The *rank* could be used to *move-ahead* to the appropriate location in the sequence

| Symbol | Code |
|--------|------|
| A      | 0    |
| B      | 10   |
| C      | 11   |

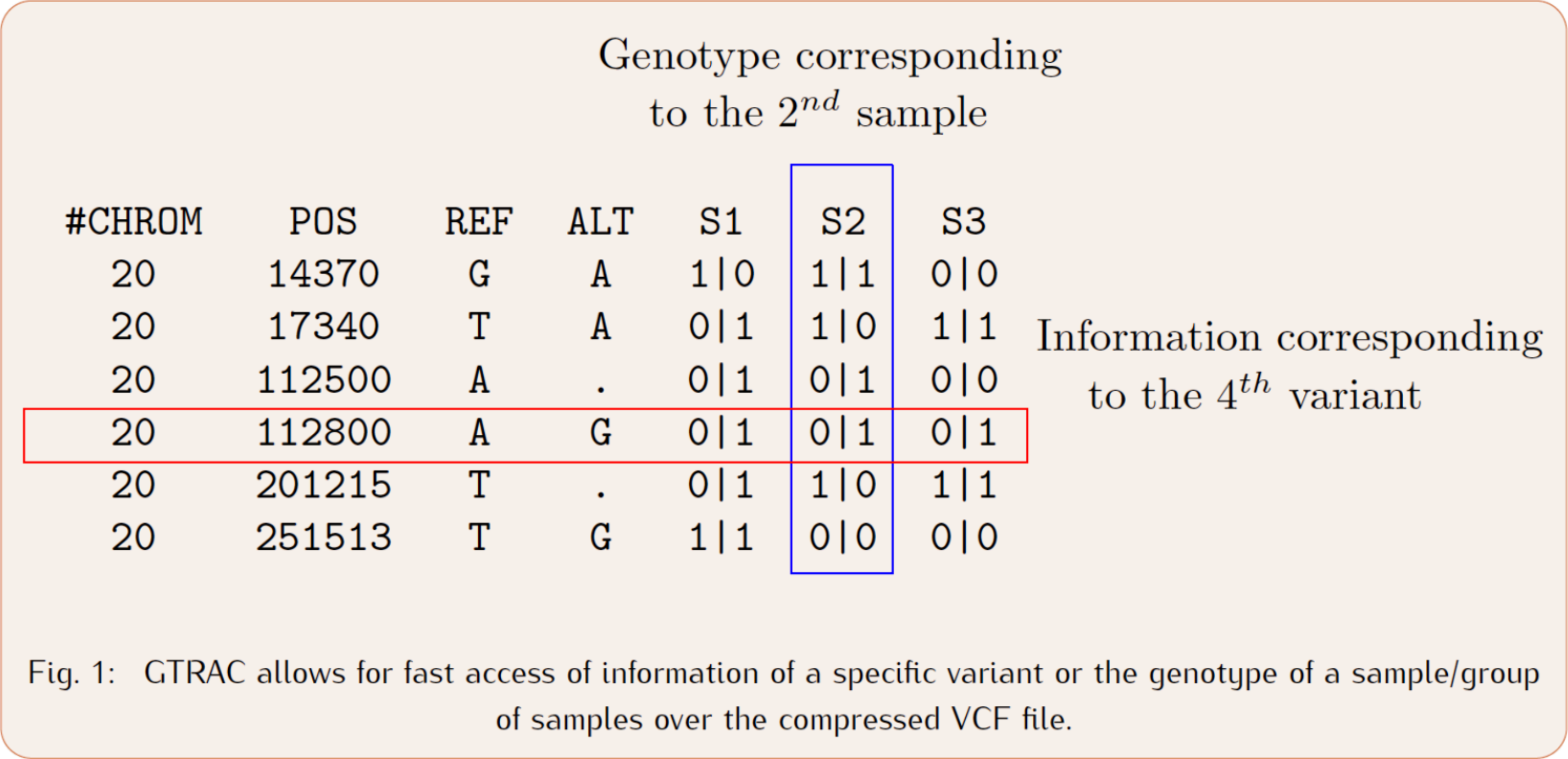
Sample sequence: **A B A A C B A ...**  
Code: **0100011100...**

# Random Access/Local decodability

**Question:** How can we modify Huffman codes to support random access?

- **In practice:**  
Combination of super-blocks & storing *rank* over smaller sub-blocks is used
- Super-blocks and sub-blocks need to be chosen carefully to get good speed/compression tradeoff

# Succinct Data Structures - random access



GTRAC: fast retrieval from compressed collections of genomic variants, <https://doi.org/10.1093/bioinformatics/btw437>



# Succinct Data Structures - FM Index

## The FM-Index: A Compressed Full-Text Index Based on the BWT

Paolo Ferragina  
Università di Pisa  
ferragina@di.unipi.it

Giovanni Manzini  
Università del Piemonte Orientale  
manzini@unipmn.it

In this talk we address the issue of indexing compressed data both from the theoretical and the practical point of view.

We start by introducing the *FM-index* data structure [2] that supports substring searches and occupies a space which is a function of the entropy of the indexed data. The key feature

<http://archive.dimacs.rutgers.edu/Workshops/BWT/ferragina.pdf>

# Succinct data structures resources:

| Category      | Method / Visual Idea | Use / Description                                    |
|---------------|----------------------|------------------------------------------------------|
| Bit Vectors   | RRR                  | Compressed bitvectors; O(1) rank/select              |
| Text Indexing | FM-index             | BWT + rank; compressed pattern search                |
| Sequences     | Wavelet Tree         | Rank/select/access on sequences efficiently          |
| Integer Sets  | Elias–Fano           | Compact representation of monotone integer sequences |



**Thank you!**